ByteMart Digital

THE INTERVIEW PREP SERIES

Spring Boot Interview Q&A

IoC & DI, beans, web layer, data & transactions, AOP, security, testing & coding

90 interview questions with concise, accurate model answers, including 6 hands-on coding challenges with working solutions — organized by topic for focused preparation.

90	6	15	Model
Questions	Coding	Sections	Answers

© 2026 ByteMart Digital · All Rights Reserved

ByteMart Digital

Spring Boot Interview Questions

First Edition · 2026

Copyright © 2026 ByteMart Digital. All rights reserved.
No part of this publication may be reproduced, distributed, or transmitted in any form or by any means — including photocopying, recording, screenshotting, or other electronic or mechanical methods — without the prior written permission of the publisher, except for brief quotations used in reviews and certain other non-commercial uses permitted by copyright law.

This product is licensed for personal use by the purchaser only. Sharing, reselling, or redistributing this file in whole or in part is strictly prohibited and constitutes a violation of copyright law.

Disclaimer
This material is provided in good faith for educational and interview-preparation purposes. While every effort has been made to ensure technical accuracy, the author and publisher assume no responsibility for errors, omissions, or outcomes arising from its use. All code examples are provided "as is".

Trademarks
Java is a registered trademark of Oracle. Spring and Spring Boot are trademarks of Broadcom. All other product names and brands are the property of their respective owners and are used for identification purposes only. This is an independent work.

Published by ByteMart Digital

Web: bytemartdigital.com
Email: contact@bytemartdigital.in
Instagram: instagram.com/bytemartdigital.in
Telegram: t.me/bytemartcodingcommunity

Contents

Core Concepts & IoC 4

Beans & Configuration 8

Web Layer & REST 11

Data & Transactions 14

AOP, Security & Observability 17

Testing & Startup 19

Spring MVC Internals 21

Caching, Async & Events 23

Data Access Deep-Dive 25

Beans, Wiring & Pitfalls 27

Resilience & Production 29

Serialization & API Design 31

Configuration & Testing Extras 33

Advanced Topics 35

Coding Challenges 38

This bank contains 90 Spring Boot interview questions across 15 sections, including 6 coding challenges with full solutions. Use the bookmarks panel to jump to any section or question.

SECTION 01

Core Concepts & IoC

Q1 What is Spring Boot, and how does it differ from the Spring Framework?

ANSWER

Spring Boot is an opinionated layer built on top of the Spring Framework that eliminates most of the configuration Spring traditionally required. Where classic Spring needed extensive XML or Java configuration and an external server, Spring Boot provides sensible defaults, bundled dependencies, and an embedded server so you can run a production-grade application with almost no setup.

- **Auto-configuration** — configures beans automatically based on the classpath.
- **Starter dependencies** — curated dependency bundles like `spring-boot-starter-web`.
- **Embedded server** — Tomcat/Jetty/Undertow inside the app; runs as a standalone JAR.
- **Actuator** — health checks, metrics, and monitoring endpoints out of the box.

> KEY IDEA
> Spring Boot is Spring plus auto-configuration, starters, an embedded server, and production tooling — convention over configuration.

Q2 Explain Inversion of Control and Dependency Injection.

ANSWER

Inversion of Control (IoC) means the framework, not your code, is responsible for creating objects and wiring them together. Instead of a class instantiating its collaborators, it declares what it needs and the container supplies them. **Dependency Injection (DI)** is the mechanism that realises IoC.

This inversion decouples classes from their dependencies' concrete implementations, which makes code easier to test (inject mocks), easier to change (swap implementations), and easier to reason about (dependencies are explicit).

```java
@Service
public class OrderService {
    private final OrderRepository repo;
    public OrderService(OrderRepository repo) {   // injected by the container
        this.repo = repo;
    }
}
```

KEY IDEA
IoC hands control of object creation and wiring to the container; DI is how the container supplies each bean its dependencies.

Q3 Which type of dependency injection is preferred, and why?

ANSWER

Constructor injection is the recommended approach. It makes dependencies explicit and mandatory (you cannot construct the object without them), allows fields to be declared `final` for immutability, and makes the class trivial to unit-test by simply passing mocks to the constructor — no Spring context required.

	Constructor	Field / Setter
Dependencies visible	yes (in signature)	hidden
final fields	yes	no
Testable without Spring	easily	harder

WATCH OUT
Avoid field injection (`@Autowired` on a field) — it hides dependencies, prevents `final` fields, and forces reflection-based test setup.

Q4 How does Spring Boot auto-configuration work?

ANSWER

Auto-configuration inspects the classpath, the beans you have already defined, and your properties, then **conditionally** configures beans using `@Conditional` annotations such as `@ConditionalOnClass` and `@ConditionalOnMissingBean`. If a JPA starter and a database driver are present, for example, it configures a `DataSource` and an `EntityManager` for you.

Crucially, it backs off whenever you define your own bean — so any default is easily overridden. The list of auto-configurations is discovered from the starters on the classpath.

NOTE
`@SpringBootApplication` is a convenience annotation combining `@EnableAutoConfiguration`, `@ComponentScan`, and `@Configuration`.

Q5 What does @SpringBootApplication do?

ANSWER

It is a meta-annotation bundling three: `@Configuration` (marks the class as a bean-definition source), `@ComponentScan` (scans the package and sub-packages for components), and `@EnableAutoConfiguration` (activates Spring Boot's auto-configuration). Placing it on the main class bootstraps the whole application.

> **WATCH OUT**
> Component scanning starts from the annotated class's package — place your main class in a root package so all your components are discovered.

Q6 What are Spring Boot starters?

ANSWER

A **starter** is a curated set of dependencies for a capability, so you add one artifact instead of hand-picking many compatible versions. `spring-boot-starter-web` pulls in Spring MVC, Jackson, and an embedded Tomcat; `spring-boot-starter-data-jpa` pulls in JPA and Hibernate. Versions are managed by the Boot parent, avoiding conflicts.

Q7 What is the difference between ApplicationContext and BeanFactory?

ANSWER

`BeanFactory` is the basic container providing dependency injection. `ApplicationContext` extends it with enterprise features — event publishing, internationalisation, annotation processing, and automatic bean post-processing — and is what Spring Boot uses. In practice you always work with an `ApplicationContext`.

Q8 What is the Spring bean lifecycle?

ANSWER

The container instantiates the bean, injects its dependencies, calls any post-processors, then runs initialisation callbacks (`@PostConstruct` or `InitializingBean`). The bean is then in use for the application's lifetime, and on shutdown the container runs destruction callbacks (`@PreDestroy` or `DisposableBean`).

© 2026 ByteMart Digital

java

```java
@Component
public class Cache {
    @PostConstruct void warmUp() { /* load initial data */ }
    @PreDestroy   void flush()  { /* persist on shutdown */ }
}
```

SECTION 02

Beans & Configuration

Q9 What is the difference between @Component, @Service, and @Repository?

ANSWER

All three are stereotype annotations that register a class as a Spring-managed bean via component scanning. They are technically interchangeable but express **intent** and can add role-specific behaviour.

Annotation	Role / behaviour
@Component	generic managed bean
@Service	business-logic layer (semantic)
@Repository	data access + exception translation

`@Repository` additionally translates persistence-specific exceptions into Spring's consistent `DataAccessException` hierarchy, which is a concrete behavioural difference, not just documentation.

Q10 What is the difference between @Bean and @Component?

ANSWER

`@Component` is placed on a class so component scanning detects and instantiates it — Spring controls its creation. `@Bean` is placed on a method inside a `@Configuration` class, where you construct and return the object yourself, giving full control. Use `@Bean` for third-party classes you cannot annotate or when construction needs custom logic.

```java
@Configuration
public class AppConfig {
    @Bean
    public RestClient restClient() {
        return RestClient.builder().baseUrl("https://api.example.com").build();
    }
}
```

Q11 What are the bean scopes in Spring?

ANSWER

The scope controls a bean's lifecycle and how many instances the container creates. The default is **singleton** — one shared instance per container. Other scopes include **prototype** (a new instance every time it is requested) and the web scopes **request**, **session**, and **application**.

> **WATCH OUT**
> Singleton beans are shared across all requests and threads, so they must be **stateless**. Storing per-request data in a singleton's fields causes race conditions and data bleeding between users.

Q12 How is configuration externalized in Spring Boot?

ANSWER

Configuration lives in `application.yml` or `application.properties` and follows a well-defined precedence: command-line arguments and environment variables override file values, and profile-specific files override the base. This lets the same build run in every environment with different settings.

Bind related properties to a typed class with `@ConfigurationProperties` for structure and validation, or inject a single value with `@Value`.

```java
@ConfigurationProperties(prefix = "app.mail")
public record MailProps(String host, int port, String from) { }
```

Q13 What is the difference between @Value and @ConfigurationProperties?

ANSWER

`@Value` injects a single property by expression — convenient for one-off values. `@ConfigurationProperties` binds an entire group of related properties onto a typed object, giving structure, IDE support, relaxed binding, and optional validation. For anything beyond a single value, prefer `@ConfigurationProperties`.

© 2026 ByteMart Digital

Q14 What are Spring profiles, and how are they used?

ANSWER

Profiles let you define environment-specific beans and configuration — typically `dev`, `test`, and `prod`. A file named `application-prod.yml` applies only when the `prod` profile is active, and `@Profile("prod")` registers a bean only under that profile. The active profile is set via a property, environment variable, or launch argument.

Q15 How do you resolve ambiguity when multiple beans match a dependency?

ANSWER

When two beans of the same type exist, Spring cannot decide which to inject and fails. Resolve it with `@Primary` to mark a default, or `@Qualifier("name")` at the injection point to name the specific bean you want.

```java
@Bean @Primary DataSource primaryDs() { ... }
@Bean @Qualifier("audit") DataSource auditDs() { ... }

public Service(@Qualifier("audit") DataSource ds) { ... }
```

Q16 What are conditional beans?

ANSWER

Conditional annotations register a bean only when a condition holds — `@ConditionalOnProperty` (a property has a value), `@ConditionalOnClass` (a class is present), `@ConditionalOnMissingBean` (no such bean exists yet). They are the foundation of auto-configuration and let you toggle features by configuration.

© 2026 ByteMart Digital

SECTION 03

Web Layer & REST

Q17 What is the difference between @Controller and @RestController?

ANSWER

`@Controller` is for traditional MVC — its methods return logical view names that a template engine renders into HTML. `@RestController` is a convenience combining `@Controller` and `@ResponseBody`, so every method's return value is serialized directly into the response body (usually JSON), which is what REST APIs need.

Q18 Explain @RequestMapping and its HTTP-specific shortcuts.

ANSWER

`@RequestMapping` maps requests to handler methods by path, HTTP method, headers, and content type. The shortcuts — `@GetMapping`, `@PostMapping`, `@PutMapping`, `@DeleteMapping`, `@PatchMapping` — are concise, method-specific forms that make intent obvious.

```java
@GetMapping("/orders/{id}")
public OrderDto get(@PathVariable Long id) { ... }

@PostMapping("/orders")
@ResponseStatus(HttpStatus.CREATED)
public OrderDto create(@RequestBody CreateOrder req) { ... }
```

Q19 What is the difference between @PathVariable, @RequestParam, and @RequestBody?

ANSWER

- **@PathVariable** — binds a value embedded in the URL path (`/orders/42` → `id=42`).
- **@RequestParam** — binds a query-string parameter (`?status=paid`) or form field.
- **@RequestBody** — deserializes the request body (typically JSON) into a Java object.

Q20 What is ResponseEntity, and when should you use it?

ANSWER

`ResponseEntity<T>` represents the full HTTP response — status code, headers, and body — giving you explicit control instead of relying on defaults. Use it when you need to return specific status codes (201, 204, 404), set headers (Location, ETag), or vary the response shape.

© 2026 ByteMart Digital

```java
return ResponseEntity.status(HttpStatus.CREATED)
    .header("Location", "/orders/" + created.id())
    .body(created);
```

Q21 How do you validate request data?

ANSWER

Annotate the request object's fields with Bean Validation constraints (`@NotBlank`, `@Email`, `@Min`), then add `@Valid` to the controller parameter. Spring validates automatically and, on failure, raises `MethodArgumentNotValidException`, which you handle globally to return a 400/422 with field-level errors.

```java
public record CreateUser(@NotBlank String name,
                         @Email String email,
                         @Min(18) int age) { }
@PostMapping("/users")
public User create(@Valid @RequestBody CreateUser req) { ... }
```

Q22 How do you handle exceptions globally in a REST API?

ANSWER

Use a class annotated with `@RestControllerAdvice` containing `@ExceptionHandler` methods. This centralises error handling across every controller, so you return consistent, structured error responses with correct status codes and never leak stack traces to clients.

```java
@RestControllerAdvice
public class ApiExceptionHandler {
    @ExceptionHandler(NotFoundException.class)
    public ResponseEntity<ErrorResponse> handle(NotFoundException e) {
        return ResponseEntity.status(404)
            .body(new ErrorResponse("not_found", e.getMessage()));
    }
}
```

© 2026 ByteMart Digital

Q23 What is the difference between a Filter and an Interceptor?

ANSWER

A **servlet Filter** operates at the servlet-container level, wrapping every request before it reaches Spring — good for cross-cutting concerns like logging, compression, and CORS. A **HandlerInterceptor** operates within Spring MVC, with access to the handler and model, and can run logic before/after controller execution. Filters are broader; interceptors are Spring-aware.

Q24 How do you handle CORS in Spring Boot?

ANSWER

Cross-Origin Resource Sharing controls which origins may call your API from a browser. Configure it per-endpoint with `@CrossOrigin`, or globally via a `WebMvcConfigurer` that adds CORS mappings — specifying allowed origins, methods, and headers. Global configuration is preferred for consistency.

Q25 What is content negotiation?

ANSWER

Content negotiation lets one endpoint serve multiple representations. The client's `Accept` header signals the format it wants, and Spring selects a matching `HttpMessageConverter` to serialize the response — JSON by default with Jackson, or XML if configured. The `Content-Type` header describes the request body's format.

SECTION 04

Data & Transactions

Q26 What is Spring Data JPA, and what does a repository give you?

ANSWER

Spring Data JPA is an abstraction over JPA/Hibernate that removes boilerplate data-access code. By declaring an interface that extends `JpaRepository`, you get CRUD, paging, and sorting for free, and Spring generates the implementation at runtime. You add behaviour with derived query methods or custom queries.

```java
public interface OrderRepository extends JpaRepository<Order, Long> {
    List<Order> findByStatus(String status);       // derived query
    Optional<Order> findByReference(String ref);
}
```

Q27 How do derived query methods work?

ANSWER

Spring Data parses the method name and generates the query from it. Keywords like `findBy`, `And`, `Or`, `OrderBy`, `Between`, and `Containing` map to query clauses. This is concise for straightforward queries, though names get unwieldy for complex ones.

```java
List<User> findByActiveTrueAndAgeGreaterThanOrderByCreatedAtDesc(int age);
```

Q28 When and how do you use @Query?

ANSWER

Use `@Query` when a query is too complex for a derived name, when you need a `JOIN FETCH`, or when you want native SQL. It supports JPQL by default and native SQL with `nativeQuery = true`, and binds parameters by name.

```java
@Query("select o from Order o join fetch o.items where o.status = :s")
List<Order> findWithItems(@Param("s") String status);
```

© 2026 ByteMart Digital

Q29 How does @Transactional work?

ANSWER

Placing `@Transactional` on a method (or class) makes Spring wrap the call in a database transaction via a proxy. The transaction commits if the method returns normally and rolls back if it throws. Within the transaction the persistence context stays open, so managed entities are tracked and changes flushed on commit.

> **WATCH OUT**
> Because it works through a proxy, calling one `@Transactional` method from another in the **same class** (self-invocation) bypasses the proxy and the transaction — a very common, subtle bug.

Q30 What are the default rollback rules for @Transactional?

ANSWER

By default, Spring rolls back on **unchecked** exceptions (`RuntimeException` and `Error`) but **commits** on checked exceptions. You can change this with `rollbackFor` and `noRollbackFor`. Many bugs stem from a checked exception silently committing a partial transaction.

```java
@Transactional(rollbackFor = Exception.class)   // roll back on checked too
public void process() throws IOException { ... }
```

Q31 What is transaction propagation?

ANSWER

Propagation defines how a transactional method behaves when called within an existing transaction. The default `REQUIRED` joins an existing transaction or starts one. `REQUIRES_NEW` always suspends the current and starts a fresh, independent transaction — useful for actions like audit logging that must commit regardless of the outer transaction's outcome.

Propagation	Behaviour
REQUIRED (default)	join or start a transaction
REQUIRES_NEW	suspend outer, start a new one
SUPPORTS	join if present, else none
MANDATORY	must run within an existing one

Q32 What is the N+1 problem in the context of Spring Data JPA?

ANSWER

When you load a list of entities and then access a lazy association on each, Spring Data issues one query for the list plus one per entity — N+1 queries — which degrades badly at scale. Fix it with a `JOIN FETCH` in a `@Query`, an `@EntityGraph`, or batch fetching.

```java
@EntityGraph(attributePaths = "items")
List<Order> findByStatus(String status);   // fetches items in one query
```

SECTION 05

AOP, Security & Observability

Q33 What is Aspect-Oriented Programming, and when is it used?

ANSWER

AOP modularises **cross-cutting concerns** — logic that would otherwise be scattered across many methods, such as logging, security checks, caching, and transactions. You define an **aspect** containing **advice** (the code to run) applied at **join points** (method executions) matched by a **pointcut** expression.

Spring implements AOP with proxies. In fact, `@Transactional` and `@Cacheable` are themselves aspects. Custom aspects let you add behaviour declaratively without touching business code.

```java
@Aspect @Component
public class TimingAspect {
    @Around("@annotation(Timed)")
    public Object time(ProceedingJoinPoint pjp) throws Throwable {
        long start = System.nanoTime();
        try { return pjp.proceed(); }
        finally { log(System.nanoTime() - start); }
    }
}
```

Q34 What are the AOP advice types?

ANSWER

- **@Before** — runs before the method.
- **@AfterReturning** — after successful completion.
- **@AfterThrowing** — when the method throws.
- **@After** — after the method regardless of outcome (finally).
- **@Around** — wraps the method; can control whether and how it runs.

Q35 How does Spring Security handle authentication and authorization?

ANSWER

Spring Security inserts a chain of servlet filters that intercept requests. **Authentication** establishes who the caller is (via a token, session, or credentials), and **authorization** decides what they may access, enforced by URL rules or method-level annotations like `@PreAuthorize`. A `SecurityFilterChain` bean configures the rules.

© 2026 ByteMart Digital

```java
@Bean
SecurityFilterChain chain(HttpSecurity http) throws Exception {
    http.authorizeHttpRequests(a -> a
            .requestMatchers("/admin/**").hasRole("ADMIN")
            .anyRequest().authenticated())
        .oauth2ResourceServer(o -> o.jwt(Customizer.withDefaults()));
    return http.build();
}
```

Q36 How is JWT-based authentication typically implemented?

ANSWER

The client authenticates once and receives a signed **JWT** containing claims (user id, roles, expiry). It sends the token in the `Authorization: Bearer` header on every request; the server validates the signature and expiry and builds the security context from the claims. Being stateless, JWTs suit horizontally-scaled and microservice systems.

> **WATCH OUT**
> JWTs cannot be easily revoked before they expire, so keep lifetimes short and pair them with refresh tokens; never put sensitive data in a token, since the payload is only encoded, not encrypted.

Q37 What is Spring Boot Actuator?

ANSWER

Actuator exposes production-ready endpoints for monitoring and managing a running application — `/actuator/health` (liveness/readiness), `/actuator/metrics`, `/actuator/info`, and more. It integrates with Micrometer to publish metrics to systems like Prometheus.

> **WATCH OUT**
> Actuator endpoints can reveal sensitive internals, so secure them and expose only the ones you need over the network.

SECTION 06

Testing & Startup

Q38 How do you test a Spring Boot application?

ANSWER

Test in layers. Use plain unit tests with mocked dependencies for business logic (no Spring context, fastest). Use slice tests for specific layers — `@WebMvcTest` for controllers with `MockMvc`, `@DataJpaTest` for repositories against an in-memory database. Reserve `@SpringBootTest`, which loads the full context, for genuine end-to-end checks.

> **TIP**
> Keep the test pyramid healthy: many fast unit tests, fewer slice tests, and a small number of full-context integration tests.

Q39 What is the difference between @Mock and @MockBean?

ANSWER

`@Mock` (from Mockito) creates a mock in a plain unit test with no Spring involvement. `@MockBean` (from Spring Boot Test) creates a mock **and** places it into the application context, replacing the real bean — used in slice or integration tests where the context is loaded.

Q40 What is MockMvc?

ANSWER

`MockMvc` lets you test controllers by simulating HTTP requests against the Spring MVC infrastructure without starting a real server. You assert on status, headers, and body, making controller tests fast and focused.

```java
mockMvc.perform(get("/orders/1"))
    .andExpect(status().isOk())
    .andExpect(jsonPath("$.status").value("PAID"));
```

Q41 What happens when a Spring Boot application starts up?

ANSWER

The `main` method calls `SpringApplication.run()`, which creates the `ApplicationContext`, runs component scanning and auto-configuration to register beans, resolves and injects all dependencies, runs initialisation callbacks, starts the embedded web server, and executes any `CommandLineRunner/ApplicationRunner` beans — after which it accepts requests.

SECTION 07

Spring MVC Internals

Q42 What is the DispatcherServlet and the request-handling flow?

ANSWER

The `DispatcherServlet` is Spring MVC's front controller — a single servlet that receives every request and orchestrates the response. It consults a `HandlerMapping` to find the controller method, invokes it through a `HandlerAdapter`, resolves the return value (a view or, for REST, a serialized body), and writes the response.

This central dispatch is what lets Spring apply cross-cutting behaviour — interceptors, argument resolvers, message converters, and exception handlers — consistently to every request.

Q43 How does Spring convert a Java object to JSON and back?

ANSWER

Spring uses `HttpMessageConverters`. For JSON, the `MappingJackson2HttpMessageConverter` wraps Jackson's `ObjectMapper`, which serializes return values and deserializes `@RequestBody` parameters. You customise behaviour with Jackson annotations or a configured `ObjectMapper` bean.

```java
public record UserDto(
    @JsonProperty("user_name") String name,
    @JsonIgnore String passwordHash) { }
```

Q44 What does @ResponseStatus do?

ANSWER

`@ResponseStatus` sets the HTTP status code for a controller method or an exception class declaratively, without building a `ResponseEntity`. Placing it on a custom exception makes Spring return that status automatically whenever the exception is thrown.

```java
@ResponseStatus(HttpStatus.NOT_FOUND)
public class NotFoundException extends RuntimeException { }
```

Q45 What is the difference between @ControllerAdvice and @RestControllerAdvice?

ANSWER

Both apply `@ExceptionHandler`, `@InitBinder`, and `@ModelAttribute` methods across controllers. `@RestControllerAdvice` adds `@ResponseBody`, so handler return values are serialized to the response body (JSON) — the right choice for REST APIs, whereas `@ControllerAdvice` is for view-based MVC.

SECTION 08

Caching, Async & Events

Q46 How does Spring's caching abstraction work?

ANSWER

Enable it with `@EnableCaching`, then annotate methods: `@Cacheable` stores and reuses results by key, `@CachePut` updates the cache, and `@CacheEvict` removes entries. Spring implements this with a proxy, delegating to a pluggable cache provider (Caffeine, Redis, etc.).

```java
@Cacheable(value = "users", key = "#id")
public User findById(Long id) { return repo.findById(id).orElseThrow(); }
```

WATCH OUT

Like `@Transactional`, caching is proxy-based — calling a cached method from within the same class bypasses the cache.

Q47 How do you run a method asynchronously with @Async?

ANSWER

Enable it with `@EnableAsync` and annotate a method with `@Async`; Spring runs it on a separate thread pool and returns immediately, optionally giving a `CompletableFuture` for the result. It is ideal for fire-and-forget work like sending emails.

WATCH OUT

`@Async` is proxy-based too — self-invocation runs synchronously — and the method should not return a plain value you need immediately.

Q48 What are Spring application events?

ANSWER

Spring provides an in-process publish/subscribe mechanism. A component publishes an event via `ApplicationEventPublisher`, and any bean with an `@EventListener` method for that event type receives it — decoupling the publisher from consumers. Events can be handled synchronously or, with `@Async`, asynchronously.

© 2026 ByteMart Digital

```java
@EventListener
public void onOrderPlaced(OrderPlacedEvent e) {
    notificationService.send(e.orderId());
}
```

Q49 What is the difference between RestTemplate, WebClient, and RestClient?

ANSWER

`RestTemplate` is the classic synchronous HTTP client, now in maintenance mode. `WebClient` is the modern reactive, non-blocking client (usable synchronously too). `RestClient` (Spring 6.1+) is a new synchronous client with a fluent API — the recommended choice for straightforward blocking calls.

SECTION 09

Data Access Deep-Dive

Q50 How is connection pooling handled in Spring Boot?

ANSWER

Spring Boot auto-configures **HikariCP** as the default connection pool, which reuses a bounded set of database connections rather than opening one per query. Pool size and timeouts are tuned via `spring.datasource.hikari.*` properties. A right-sized pool is critical — too small starves throughput, too large overwhelms the database.

Q51 How do you handle database schema migrations?

ANSWER

Use a migration tool — **Flyway** or **Liquibase** — which Spring Boot auto-configures. Versioned migration scripts run in order on startup, giving a repeatable, version-controlled schema across environments. This is safer than letting Hibernate auto-generate DDL, which should never be used in production.

> **WATCH OUT**
> Never set `ddl-auto=update` in production — it can silently alter or fail to migrate schemas; use Flyway/Liquibase instead.

Q52 What is JPA auditing, and how do you enable it?

ANSWER

JPA auditing automatically populates metadata fields — created/modified timestamps and users. Enable it with `@EnableJpaAuditing` and annotate entity fields with `@CreatedDate`, `@LastModifiedDate`, and (with an `AuditorAware` bean) `@CreatedBy`.

```java
@EntityListeners(AuditingEntityListener.class)
@Entity
public class Order {
    @CreatedDate private Instant createdAt;
    @LastModifiedDate private Instant updatedAt;
}
```

Q53 How do you build dynamic queries in Spring Data JPA?

ANSWER

For queries whose criteria vary at runtime, derived methods and static `@Query` are too rigid. Use the **Specification** API (JPA Criteria under the hood) or Querydsl to compose predicates programmatically, combining only the filters the caller actually supplied.

```java
Specification<Order> spec = Specification.where(null);
if (status != null) spec = spec.and(OrderSpecs.hasStatus(status));
if (min != null)    spec = spec.and(OrderSpecs.totalAtLeast(min));
repo.findAll(spec, pageable);
```

SECTION 10

Beans, Wiring & Pitfalls

Q54 How does Spring handle circular dependencies?

ANSWER

If two singleton beans depend on each other via constructor injection, Spring cannot create either first and throws a `BeanCurrentlyInCreationException` at startup. The real fix is to redesign — extract the shared logic into a third bean, or break the cycle. As a last resort, setter or `@Lazy` injection can defer one side.

> **WATCH OUT**
> A circular dependency is usually a design smell signalling that responsibilities are tangled — prefer refactoring over patching it with `@Lazy`.

Q55 What does @Lazy do?

ANSWER

`@Lazy` defers a bean's creation until it is first actually needed, rather than at startup. It can speed up startup for rarely-used heavyweight beans and can help break a circular dependency, but overuse hides initialisation errors that would otherwise surface immediately.

Q56 What is the difference between @PostConstruct and a constructor?

ANSWER

The constructor runs before dependency injection completes, so injected fields may not be ready. `@PostConstruct` runs after all dependencies are injected, making it the right place for initialisation logic that relies on injected collaborators.

Q57 How do you expose a custom Actuator health indicator?

ANSWER

Implement the `HealthIndicator` interface as a Spring bean; Actuator aggregates it into `/actuator/health`. This lets you report the health of a critical downstream dependency so orchestration systems can react.

© 2026 ByteMart Digital

java

```java
@Component
public class QueueHealth implements HealthIndicator {
    public Health health() {
        return queue.isConnected() ? Health.up().build()
                                   : Health.down().build();
    }
}
```

SECTION 11

Resilience & Production

Q58 How do you implement retries in Spring?

ANSWER

Spring Retry provides declarative retries. Enable it with `@EnableRetry` and annotate a method with `@Retryable`, specifying which exceptions to retry, the maximum attempts, and a backoff. A `@Recover` method handles the case where all attempts fail.

java

```
@Retryable(retryFor = IOException.class, maxAttempts = 3,
           backoff = @Backoff(delay = 500, multiplier = 2))
public String call() throws IOException { return client.fetch(); }

@Recover
public String recover(IOException e) { return "fallback"; }
```

Q59 How do you configure graceful shutdown?

ANSWER

Graceful shutdown lets in-flight requests finish before the application stops. Enable it with `server.shutdown=graceful` and set a timeout via `spring.lifecycle.timeout-per-shutdown-phase`. This is important in Kubernetes, where pods receive a termination signal and must drain cleanly.

Q60 How do you handle file uploads?

ANSWER

Accept a `MultipartFile` parameter annotated with `@RequestParam` in a controller mapped to a multipart request. Configure size limits via `spring.servlet.multipart.max-file-size`. Stream large files rather than loading them fully into memory.

java

```
@PostMapping("/upload")
public String upload(@RequestParam MultipartFile file) throws IOException {
    file.transferTo(targetPath);
    return file.getOriginalFilename();
}
```

Q61 How is logging configured in Spring Boot?

ANSWER

Spring Boot uses SLF4J with Logback by default. Set levels per package in `application.yml` (`logging.level.com.bytemart=DEBUG`), configure file output and rotation, or drop in a `logback-spring.xml` for full control. Structured (JSON) logging is common for aggregation in production.

Q62 What is the difference between liveness and readiness probes?

ANSWER

Actuator exposes both. **Liveness** indicates whether the application is running — if it fails, the platform restarts the instance. **Readiness** indicates whether it can serve traffic — if it fails, the platform stops routing requests to it without restarting. They map to Kubernetes probes for correct lifecycle handling.

SECTION 12

Serialization & API Design

Q63 How do you customise JSON serialization with Jackson?

ANSWER

Control field mapping with annotations — `@JsonProperty` (rename), `@JsonIgnore` (omit), `@JsonFormat` (dates), `@JsonInclude` (skip nulls) — or configure a global `ObjectMapper` bean for defaults like snake_case naming and ISO dates. For complex cases, write a custom serializer.

```java
@JsonInclude(JsonInclude.Include.NON_NULL)
public record ProductDto(
    @JsonProperty("product_id") Long id,
    @JsonFormat(pattern = "yyyy-MM-dd") LocalDate releaseDate) { }
```

Q64 Why should you use DTOs instead of exposing entities?

ANSWER

Returning JPA entities directly couples your API to your database schema, risks exposing sensitive fields, and can trigger lazy-loading serialization errors. A **DTO** is a purpose-built shape for the API that decouples it from persistence, controls exactly what is exposed, and lets the two evolve independently.

> **WATCH OUT**
> Serializing a lazy entity outside a transaction throws `LazyInitializationException` — mapping to a DTO within the transaction avoids this entirely.

Q65 What are projections in Spring Data?

ANSWER

Projections let a repository return only a subset of an entity's fields, as an interface or a DTO, instead of the full entity. This reduces data transfer and avoids loading unneeded columns — a lightweight alternative to full entities for read-heavy endpoints.

```java
interface OrderSummary { Long getId(); String getStatus(); }
List<OrderSummary> findByCustomerId(Long id);   // fetches only id + status
```

© 2026 ByteMart Digital

Q66 How do you version a Spring Boot REST API?

ANSWER

Version only for breaking changes and evolve additively otherwise. The common approaches are URI versioning (`/v2/orders`), header versioning, or content-type versioning. URI versioning is the most visible and cache-friendly, so it is the frequent default.

Q67 How do you implement pagination in a Spring Boot endpoint?

ANSWER

Accept a `Pageable` (or page/size params) and pass it to a Spring Data repository, which returns a `Page` carrying the content plus total counts and navigation metadata. Cap the page size to protect the server from unbounded requests.

```java
@GetMapping("/orders")
public Page<OrderDto> list(@RequestParam(defaultValue="0") int page,
                           @RequestParam(defaultValue="20") int size) {
    return repo.findAll(PageRequest.of(page, Math.min(size, 100)))
               .map(this::toDto);
}
```

SECTION 13

Configuration & Testing Extras

Q68 What is the difference between @Validated and @Valid?

ANSWER

`@Valid` is the standard Bean Validation annotation for cascading validation on a request body. `@Validated` is Spring's variant that additionally supports **validation groups** (validating different constraints in different contexts) and enables method-level validation on beans (e.g. validating `@RequestParam` values).

Q69 How do you write integration tests with a real database using Testcontainers?

ANSWER

Testcontainers spins up a real database in a Docker container for the test, giving production-like fidelity that in-memory databases lack. Combined with `@SpringBootTest` or `@DataJpaTest`, it verifies actual SQL behaviour, migrations, and dialect-specific features.

Q70 What does @ConditionalOnProperty enable?

ANSWER

It registers a bean or configuration only when a property has a specified value — letting you toggle features on or off purely through configuration, without code changes. It is widely used for optional integrations and feature flags.

```java
@Bean
@ConditionalOnProperty(name = "features.audit", havingValue = "true")
public AuditService auditService() { return new AuditService(); }
```

Q71 How do you externalise and override properties in different environments?

ANSWER

Spring Boot resolves properties in a fixed precedence: command-line arguments, then OS environment variables, then profile-specific files, then the base `application.yml`. This lets a single artifact behave differently per environment — for example, injecting production credentials via environment variables that override file defaults.

Q72 What is the purpose of @SpringBootTest's webEnvironment options?

ANSWER

It controls how the web layer is set up for a full-context test: `MOCK` (default — a mock servlet environment with `MockMvc`), `RANDOM_PORT` or `DEFINED_PORT` (a real embedded server for end-to-end tests with `TestRestTemplate` or `WebTestClient`), or `NONE` (no web environment).

SECTION 14

Advanced Topics

Q73 What is the difference between fixedRate and fixedDelay in @Scheduled?

ANSWER

`fixedRate` starts each execution at a fixed interval measured from the start of the previous run, so runs can overlap if one is slow. `fixedDelay` waits a fixed gap after the previous run finishes, so executions never overlap. Use `fixedDelay` when a task must not run concurrently with itself, and cron for calendar-based schedules.

Q74 What is the difference between CommandLineRunner and ApplicationRunner?

ANSWER

Both run once after the application context is ready, ideal for startup tasks. `CommandLineRunner` receives the raw string arguments, while `ApplicationRunner` receives a parsed `ApplicationArguments` object with convenient access to options and non-option arguments.

Q75 What is ProblemDetail (RFC 7807), and why use it?

ANSWER

`ProblemDetail` (Spring 6+) is a standardised, machine-readable error format with fields like `type`, `title`, `status`, `detail`, and `instance`. Using a consistent, standardised error body across your API makes client-side error handling predictable and interoperable.

```java
ProblemDetail pd = ProblemDetail.forStatusAndDetail(
    HttpStatus.NOT_FOUND, "Order 42 was not found");
pd.setType(URI.create("https://api.bytemart.com/errors/not-found"));
return pd;
```

Q76 How do you stream data to a client (SSE)?

ANSWER

For server-sent events, return an `SseEmitter` (or a reactive `Flux`) from a controller; the connection stays open and the server pushes events as they occur. It suits live feeds like notifications or progress updates without the overhead of full WebSockets.

Q77 What is a BeanPostProcessor?

ANSWER

A `BeanPostProcessor` is an extension point that lets you intercept every bean just before and after its initialisation, to inspect or wrap it (for example, creating proxies). Much of Spring's own machinery — annotation processing, AOP proxy creation — is built on post-processors.

Q78 How do you build a custom Spring Boot starter?

ANSWER

Create an auto-configuration class guarded by `@Conditional` annotations, register it in `META-INF/spring/...AutoConfiguration.imports`, and package it with its dependencies. Consumers then just add your starter and get the beans configured automatically — the same mechanism Boot's own starters use.

Q79 How is distributed tracing added to a Spring Boot service?

ANSWER

Micrometer Tracing (with an OpenTelemetry or Zipkin bridge) auto-instruments incoming requests and outgoing clients, propagating a trace and span id across service boundaries. Correlating these ids in logs lets you follow one request through many services — essential observability for microservices.

Q80 What is the difference between readOnly and a normal transaction?

ANSWER

`@Transactional(readOnly = true)` hints that a transaction only reads data. This lets the persistence provider skip dirty checking and flushing, and can route the call to a read replica, improving performance for query-only methods. It is a good default for service methods that never write.

Q81 How do you handle optimistic locking in a Spring Data application?

ANSWER

Add a `@Version` field to the entity; JPA checks and increments it on each update. If a concurrent transaction has changed the row, the save fails with `OptimisticLockingFailureException`, which you catch to retry or report a conflict (HTTP 409) — preventing lost updates without database locks.

© 2026 ByteMart Digital

Q82 What is the difference between spring.jpa.hibernate.ddl-auto values?

ANSWER

This controls schema generation: `none` (no action — recommended with migrations), `validate` (verify the schema matches entities), `update` (apply incremental changes — unsafe for production), and `create/create-drop` (rebuild each run — for tests only).

> **WATCH OUT**
> Use `validate` or `none` with Flyway/Liquibase in production; never `update` or `create`.

Q83 How do you internationalise messages in Spring Boot?

ANSWER

Define message bundles (`messages.properties`, `messages_hi.properties`) and inject a `MessageSource`. Spring resolves the locale from the request (via a `LocaleResolver`) and returns the matching translation, so responses and validation messages adapt to the user's language.

Q84 What is the difference between an embedded and an external server deployment?

ANSWER

By default Spring Boot packages an **embedded** server inside an executable JAR — you run it with `java -jar`, which suits containers and cloud deployment. Alternatively you can build a WAR and deploy to an **external** servlet container, but the embedded, self-contained model is now the standard and preferred approach.

SECTION 15

Coding Challenges

Q85 Write a REST controller with full CRUD endpoints for an Order resource.

</> CODE

SOLUTION

Use `@RestController` with a base path, map each HTTP method to the matching operation, inject the service via the constructor, and return appropriate status codes through `ResponseEntity`.

java

```java
@RestController
@RequestMapping("/orders")
public class OrderController {
    private final OrderService service;
    public OrderController(OrderService service) { this.service = service; }

    @GetMapping("/{id}")
    public OrderDto get(@PathVariable Long id) { return service.get(id); }

    @GetMapping
    public List<OrderDto> list() { return service.findAll(); }

    @PostMapping
    public ResponseEntity<OrderDto> create(@Valid @RequestBody CreateOrder req) {
        return ResponseEntity.status(201).body(service.create(req));
    }

    @PutMapping("/{id}")
    public OrderDto update(@PathVariable Long id, @RequestBody UpdateOrder req) {
        return service.update(id, req);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        service.delete(id);
        return ResponseEntity.noContent().build();
    }
}
```

Q86 Write a global exception handler returning consistent error responses.

</> CODE

SOLUTION

A `@RestControllerAdvice` centralises error handling: map each exception type to a status code and a structured body, including field errors for validation failures.

```java
@RestControllerAdvice
public class ApiExceptionHandler {

    @ExceptionHandler(NotFoundException.class)
    public ResponseEntity<ErrorResponse> notFound(NotFoundException e) {
        return ResponseEntity.status(404)
            .body(new ErrorResponse("not_found", e.getMessage()));
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse> invalid(MethodArgumentNotValidException e) {
        String msg = e.getBindingResult().getFieldErrors().stream()
            .map(f -> f.getField() + ": " + f.getDefaultMessage())
            .collect(Collectors.joining(", "));
        return ResponseEntity.status(422)
            .body(new ErrorResponse("validation_error", msg));
    }
}
```

Q87 Write a validated request model and bind it in a controller.

</> CODE

SOLUTION

Annotate the fields of a record with Bean Validation constraints and add `@Valid` to the parameter; failures are then handled by the global advice above.

```java
public record CreateUser(
    @NotBlank String name,
    @Email String email,
    @Min(18) int age) { }

@PostMapping("/users")
public ResponseEntity<UserDto> create(@Valid @RequestBody CreateUser req) {
    return ResponseEntity.status(201).body(service.create(req));
}
```

Q88 Write a service using constructor-injected repositories with a transaction.

</> CODE

SOLUTION

Inject repositories through the constructor, mark the state-changing method `@Transactional` so it commits or rolls back atomically, and keep the class stateless.

```java
@Service
public class TransferService {
    private final AccountRepository accounts;
    public TransferService(AccountRepository accounts) { this.accounts = accounts; }

    @Transactional
    public void transfer(Long from, Long to, BigDecimal amount) {
        Account a = accounts.findById(from).orElseThrow();
        Account b = accounts.findById(to).orElseThrow();
        a.debit(amount);
        b.credit(amount);
        // dirty checking flushes both on commit; any exception rolls back
    }
}
```

Q89 Bind typed configuration with @ConfigurationProperties and use it.

</> CODE

SOLUTION

Define a properties record, enable it, and inject it where needed. This gives structured, validated, environment-specific configuration.

```java
@ConfigurationProperties(prefix = "payment")
public record PaymentProps(String apiUrl, String apiKey, int timeoutMs) { }

@Configuration
@EnableConfigurationProperties(PaymentProps.class)
class PaymentConfig {
    @Bean RestClient paymentClient(PaymentProps props) {
        return RestClient.builder().baseUrl(props.apiUrl()).build();
    }
}
```

> **NOTE**
> In `application.yml`: `payment.api-url`, `payment.api-key`, `payment.timeout-ms` — relaxed binding maps the kebab-case keys.

Q90 Write a scheduled task that runs periodically.

</> CODE

SOLUTION

Enable scheduling on a configuration class, then annotate a method with `@Scheduled` using a fixed rate or a cron expression. The method must take no arguments.

java

```java
@Configuration
@EnableScheduling
public class Jobs {
    @Scheduled(cron = "0 0 2 * * *")   // every day at 02:00
    public void nightlyCleanup() {
        // purge stale records
    }
}
```